

第二十八讲：InnoDB 的 RedoLog 数据结构设计

知春路遇上八里桥

<2024-10-15 Tue>



1 RedoLog 引入

2 LogFile

3 RedoLog



1

RedoLog 引入



RedoLog 的意义

- mini-transaction (mtr) 保证了一组对 Page 的读写操作的原子性
 - ▶ InnoDB 通过记录 UndoLog¹ 实现
 - ▶ 一个 mtr 可能修改多个 Page, 当系统发生宕机后 RedoLog 会出现以下两种情况
 - ① 要么所有 Page 的修改都提交
 - ② 要么所有 Page 的修改都不提交
 - ▶ mtr 没有 rollback
- transaction (事务) 保证了一组对数据页 (INDEX) 修改的原子性
 - ▶ InnoDB 通过记录 UndoLog² 保证
- RedoLog 和 UndoLog 应用的范围不同
 - ▶ RedoLog 设计支持 512 字节, 刚好对应于磁盘的一个扇区
 - ▶ UndoLog 主要针对于一行数据 (UndoRecord)
 - ① UndoRecForInsert 插入一行数据
 - ② UndoRecForUpdate 修改一行数据

¹<https://dev.mysql.com/doc/refman/8.0/en/innodb-undo-logs.html>

²<https://dev.mysql.com/doc/refman/8.0/en/innodb-redo-log.html>



RedoLog 的设计

- Physiological Logging 日志记录³
 - ▶ Logical Logging 的优点是 数据量小
 - ▶ Physical Logging 的优点是基于 Page 的操作 幂等
 - ▶ InnoDB 采取了 Physiological Logging 的方式，来兼得二者的优势
 - ▶ 以 Page 为单位，但在 Page 内以逻辑的方式记录
- RedoLog 还能保证 UndoLog 的完整性，系统崩溃后会使用 RedoLog 来恢复 UndoLog
 - ▶ MySQL 8.0.30 前: ib_logfile0 和 ib_logfile1 循环记录
 - ▶ MySQL 8.0.30 后: innodb_log_group_home_dir 和 innodb_redo_log_capacity
 - ① #innodb_redo 目录下建立 32 个 RedoLog 文件
 - ② #ib_redo595 存储 RedoLog 数据 (ordinary)
 - ③ #ib_redo603_tmp 尚未写入采用 _tmp 后缀 (spare)

³https://dev.mysql.com/doc/dev/mysql-server/8.0.37/PAGE_INNODB_REDO_LOG.html



LSN 和检查点

- LSN (log sequence number) 用于记录日志序号
 - ▶ 它是全局唯一递增的 unsigned long long 类型整数
 - ▶ Page 在修改时, 会将对应的 REDO 记录的 LSN 记录在 Page 上
 - ▶ 恢复重放 REDO 时, 就可以来判断跳过已经应用的 REDO, 从而实现重放的幂等
- Checkpoint 检查点, 当发生 Checkpoint 时, 会将脏页 (数据脏页和日志脏页) 写回磁盘
- RedoLog 的 LSN 范围

```
mysql> SELECT FILE_NAME, START_LSN, END_LSN FROM performance_schema.innodb_redo_log_files;  
+-----+-----+-----+  
| FILE_NAME          | START_LSN | END_LSN |  
+-----+-----+-----+  
| ./#innodb_redo/#ib_redo1191 | 3900237824 | 3903512576 |  
+-----+-----+-----+  
1 row in set (0.01 sec)
```



2

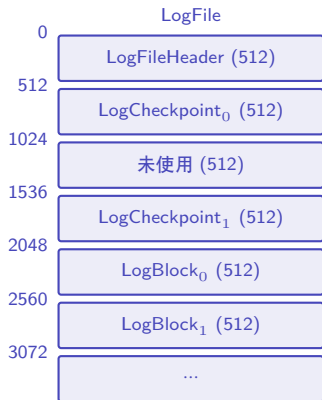
LogFile



LogFile

- LogFileHeader Redo 日志文件的头
- LogCheckpoint Redo 日志文件的检查点
- LogBlock Redo 日志文件的记录信息体
- 源码见 `storage/innobase/include/log0constants.h`

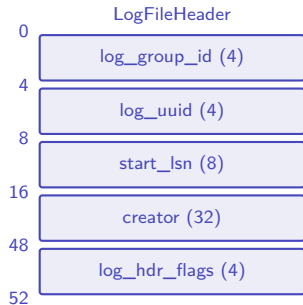
```
161  /** First checkpoint field in the log header. We write alternately to  
162  the checkpoint fields when we make new checkpoints. This field is only  
163  defined in the first log file. */  
164  constexpr os_offset_t LOG_CHECKPOINT_1 = OS_FILE_LOG_BLOCK_SIZE;  
165  
166  /** Log Encryption information in redo log header. */  
167  constexpr os_offset_t LOG_ENCRYPTION = 2 * OS_FILE_LOG_BLOCK_SIZE;  
168  
169  /** Second checkpoint field in the header of the first log file. */  
170  constexpr os_offset_t LOG_CHECKPOINT_2 = 3 * OS_FILE_LOG_BLOCK_SIZE;  
171  
172  /** Size of log file's header. */  
173  constexpr os_offset_t LOG_FILE_HDR_SIZE = 4 * OS_FILE_LOG_BLOCK_SIZE;
```



LogFileHeader

- log_group_id 日志分组 ID
- log_uuid UUID
- start_lsn 起始 LSN 编码
- creator 创建人
- log_hdr_flags 日志头信息
- 源码见 `storage/innobase/include/log0constants.h`

```
177  /** Log file header format identifier (32-bit unsigned big-endian integer).  
178  This used to be called LOG_GROUP_ID and always written as 0,  
179  because InnoDB never supported more than one copy of the redo log. */  
180  constexpr os_offset_t LOG_HEADER_FORMAT = 0;  
181  
182  /** Offset within the log file header, to the field which stores the log_uuid.  
183  The log_uuid is chosen after a new data directory is initialized, and allows  
184  to detect situation, in which some of log files came from other data directory  
185  (detection is performed on startup, before starting recovery). */  
186  constexpr uint32_t LOG_HEADER_LOG_UUID = 4;  
187  
188  /** LSN of the start of data in this log file (with format version 1 and 2). */  
189  constexpr os_offset_t LOG_HEADER_START_LSN = 8;
```



LogCheckpoint

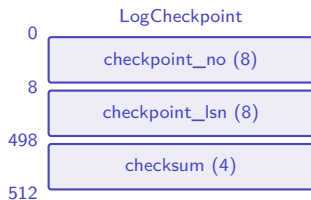
- checkpoint_no 检查点编号
- checkpoint_lsn 检查点 LSN
- checksum 校验码
- 源码见 `storage/innobase/include/log0constants.h`

```
238  /* Offsets inside the checkpoint pages since 8.0.30 redo format. */
239
240  /** Checkpoint lsn. Recovery starts from this lsn and searches for the first
241      log record group that starts since then. */
242  constexpr os_offset_t LOG_CHECKPOINT_LSN = 8;
```

- 数据库 crash recovery 时会从 LogCheckpoint 中获取上一个检查点

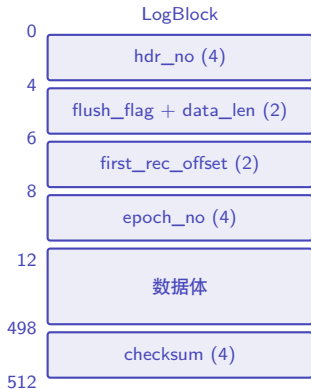
```
mysql> show engine innodb status\G
```

```
---
LOG
---
Log sequence number           3902859097
Log buffer assigned up to    3902859097
Log buffer completed up to   3902859097
Log written up to            3902859097
Log flushed up to            3902859097
Added dirty pages up to      3902859097
Pages flushed up to          3902859097
Last checkpoint at           3902859097
```



LogBlock

- RedoLog 是连续的 OS_FILE_LOG_BLOCK_SIZE (512) 字节块
 - 包含 LOG_BLOCK_HDR_SIZE (12) 字节头
 - 包含 LOG_BLOCK_TRL_SIZE (4) 字节块尾, 记录 checksum
- LogBlock 头信息
 - hdr_no 头编号
 - flush_flag flush 标识
 - epoch_no 检查点编号
 - data_len LogBlock 的数据长度, 在区间 [12,512] 中
 - first_rec_offset 指向第一个 LogBlock 数据体
- LogBlock 尾信息, 只有一个 checksum 值
- 源码见 [storage/innobase/include/log0constants.h](#)



```
246  /** Offset to hdr_no, which is a log block number and must be > 0.
247  It is allowed to wrap around at LOG_BLOCK_MAX_NO.
248  In older versions of MySQL the highest bit (LOG_BLOCK_FLUSH_BIT_MASK) of hdr_no
249  is set to 1, if this is the first block in a call to write. */
250  constexpr uint32_t LOG_BLOCK_HDR_NO = 0;
251
252  /** Mask used to get the highest bit in the hdr_no field.
253  In the older MySQL versions this bit was used to mark first block in a write.*/
254  constexpr uint32_t LOG_BLOCK_FLUSH_BIT_MASK = 0x80000000UL;
```



3

RedoLog



RedoLog 作用范围

- 源码见 `storage/innobase/include/mtr0types.h`

```
63 enum mlog_id_t {
64     /** if the mtr contains only one log record for one page,
65     i.e., write_initial_log_record has been called only once,
66     this flag is ORed to the type of that first log record */
67     MLOG_SINGLE_REC_FLAG = 128,
68
69     /** one byte is written */
70     MLOG_1BYTE = 1,
```

- 作用于 Page (页)

- ▶ 修改几字节 MLOG_1BYTE, MLOG_2BYTE, MLOG_4BYTE, MLOG_8BYTE
- ▶ 操作记录 MLOG_REC_INSERT, MLOG_REC_UPDATE_IN_PLACE

- 作用于 Space (表空间)

- ▶ MLOG_FILE_CREATE, MLOG_FILE_RENAME, MLOG_FILE_DELETE

- 提供额外信息的 Redo 日志

- ▶ MLOG_DUMMY_RECORD, MLOG_MULTI_REC_END



MLOG_nBYTE 类型的 RedoLog

- MLOG_nBYTE 修改页面中的若干字节
- page_offset 页面内偏移
- value 修改后的值
- 源码见 [storage/innobase/mtr/mtr0log.cc](#) 中 `mlog_parse_nbytes(...)` 函数

```
182 offset = mach_read_from_2(ptr);
183 ptr += 2;
184
185 if (offset >= UNIV_PAGE_SIZE) {
186     recv_sys->found_corrupt_log = true;
187
188     return (nullptr);
189 }
190
191 if (type == MLOG_8BYTES) {
192     dval = mach_u64_parse_compressed(&ptr, end_ptr);
```

MLOG_nBYTE

page_offset (2)

value (compressed)



MLOG_FILE_DELETE 类型的 RedoLog

- MLOG_FILE_DELETE 删除表空间文件
- length 文件名字符串长度
- file_name 文件名
- 源码见 `storage/innobase/fil/fil0fil.cc` 中 `fil_tablespace_redo_delete(...)` 函数

```
10490  /* Where 2 = len (uint16_t). */
10491  if (end <= ptr + 2) {
10492      return nullptr;
10493  }
10494
10495  ulint len = mach_read_from_2(ptr);
10496
10497  ptr += 2;
10498
10499  /* Do we have the full/valid file name. */
10500  if (end < ptr + len || len < 5) {
10501      if (len < 5) {
10502          char name[6];
```

MLOG_FILE_DELETE

length (2)

file_name (length)



MLOG_REC_UPDATE_IN_PLACE 类型的 RedoLog

- MLOG_REC_UPDATE_IN_PLACE 原地更新一条记录
- index_info 兼容性质的索引信息, 见 RedoLogIndexInfo
- mode_flags B+ 树的一些标记符
- trx_id_pos 事务 ID 在索引中的位置
- roll_ptr 回滚指针 / trx_id 事务 ID
- offset 页面的偏移值
- n_fields 更新字段数量 / upd_fields 更新字段列表
- 源码见 [storage/innobase/btr/btr0cur.cc](#) 中
btr_cur_parse_update_in_place(...) 函数

```
3180  /** Parses a redo log record of updating a record in-place.
3181  @return end of log record or NULL */
3182  byte *btr_cur_parse_update_in_place(
3183      byte *ptr,                /*!< in: buffer */
3184      byte *end_ptr,            /*!< in: buffer end */
3185      page_t *page,             /*!< in/out: page or NULL */
3186      page_zip_des_t *page_zip, /*!< in/out: compressed page, or NULL */
3187      dict_index_t *index)       /*!< in: index corresponding to page */
3188  {
```

MLOG_REC_UPDATE_IN_PLACE

index_info (?)
mode_flags (1)
trx_id_pos (compressed)
roll_ptr (7)
trx_id (compressed)
offset (2)
info_bits (1)
n_fields (compressed)
upd_fields (?)



RedoLogIndexInfo

- index_log_version 当前日志格式的版本
- index_flags 日志标记, COMPACT, VERSION, INSTANT
- n 索引字段数量, n_uniq 索引 key 数量
- inst_cols INSTANT DDL 添加前的列数量
- index_fields 字段的信息: 字段长度, 是否可空, 是否为定长
- 源码见 [storage/innobase/mtr/mtr0log.cc](#) 中
mlog_parse_index(...) 函数

```
1195 byte *mlog_parse_index(byte *ptr, const byte *end_ptr, dict_index_t **in  
1196     /* Read the 1 byte for index log version */  
1197     uint8_t index_log_version = 0;  
1198     ptr = parse_index_log_version(ptr, end_ptr, index_log_version);  
1199     if (ptr == nullptr) {  
1200         return nullptr;  
1201     }  
1202  
1203     byte *ret = nullptr;  
1204     switch (index_log_version) {  
1205         case INDEX_LOG_VERSION_CURRENT:  
1206             ret = mlog_parse_index_v1(ptr, end_ptr, index);  
1207             break;
```

RedoLogIndexInfo

index_log_version (1)

index_flags (1)

n (2)

n_uniq (2)

inst_cols (2)

index_fields (2 × n)



Redo 日志数据分析

```
data/#innodb_redo » ibr \#ib_redo1191 redo
```

```
RedoRecordTypes Statistics:
```

```
    MLOG_1BYTE => 27
    MLOG_2BYTES => 28
    MLOG_4BYTES => 19
    MLOG_8BYTES => 110
MLOG_REC_SEC_DELETE_MARK => 33
    MLOG_UNDO_INSERT => 4762
    MLOG_UNDO_ERASE_END => 8
    MLOG_UNDO_HDR_REUSE => 12
    MLOG_UNDO_HDR_CREATE => 8
    MLOG_INIT_FILE_PAGE2 => 1
    MLOG_TABLE_DYNAMIC_META => 4
    MLOG_REC_INSERT => 373
MLOG_REC_CLUST_DELETE_MARK => 37
    MLOG_REC_DELETE => 57
    MLOG_REC_UPDATE_IN_PLACE => 483
    UNDEF => 422
```

```
data/#innodb_redo »
```

- 查看 redo 数据文件中所有 RedoLog 类型
- 修改页面的 RedoLog
 - ▶ MLOG_1BYTE
 - ▶ MLOG_2BYTES
 - ▶ MLOG_4BYTES
 - ▶ MLOG_8BYTES
- 修改 Undo 页面的 RedoLog
 - ▶ MLOG_UNDO_INSERT
 - ▶ MLOG_UNDO_ERASE_END
 - ▶ MLOG_UNDO_HDR_REUSE
 - ▶ MLOG_UNDO_HDR_CREATE
- 修改记录的 RedoLog
 - ▶ MLOG_REC_INSERT
 - ▶ MLOG_REC_CLUST_DELETE_MARK
 - ▶ MLOG_REC_DELETE
 - ▶ MLOG_REC_UPDATE_IN_PLACE



写入 RedoLog

- 源码见 `storage/innobase/include/mtr0log.ic` 中

```
17  /** Writes the initial part of a log record (3..11 bytes).  
18  If the implementation of this function is changed, all  
19  size parameters to mlog_open() should be adjusted accordingly!  
20  @return new value of log_ptr */  
21  static inline byte *mlog_write_initial_log_record_fast(  
22      const byte *ptr, /*!< in: pointer to (inside) a buffer  
23      frame holding the file page where  
24      modification is made */  
25      mlog_id_t type, /*!< in: log item type: MLOG_1BYTE, ... */  
26      byte *log_ptr, /*!< in: pointer to mtr log which has  
27      been opened */  
28      mtr_t *mtr) /*!< in/out: mtr */  
29  {  
30      const byte *page;  
31      space_id_t space;  
32      page_no_t offset;  
33  
34      ut_ad(log_ptr);  
35      ut_d(mtr->memo_modify_page(ptr));  
36  
37      page = (const byte *)ut_align_down(ptr, UNIV_PAGE_SIZE);  
38      space = mach_read_from_4(page + FIL_PAGE_ARCH_LOG_NO_OR_SPACE_ID);  
39      offset = mach_read_from_4(page + FIL_PAGE_OFFSET);
```



结束

